

12. Iterationen

Problem: Eine bestimmte Anzahl von Primzahlen (Liste von Primzahlen) soll multipliziert werden. Die Primzahlen sind in einer also in einer Liste gespeichert.

```
1 primzahlen = [2,3,5,7,11,13]
2
3 product = primzahlen[0]* primzahlen[1]*primzahlen[2]*primzahlen[3]*primzahlen[4]*primzahlen[5]
4 print(product)
```

30030

\$\$\$ Dieser Code ist schreibaufwendig und unübersichtlich

\$\$\$ Umständlich, bei großen Listen

andere Lösung: Iterationen

12.1 for-Schleife

```
1 primzahlen = [2,3,5,7,11,13]
2 product = 1
3 for number in primzahlen:
4     product = product * number #same as product *= number
5
6 print (product)
```

30030

Syntax

```
1 for var in expresssion:
2     block
```

- Schlüsselworte sind for und in

- 1. Zeile heißt Schleifenkopf
- 2. Zeile Schleifenrumpf block mit einer oder mehreren Anweisungen
- Schleifenvariable var im Schleifenkopf
- Schleifeniteration ist ein Durchlauf (Ausführung eines Schleifenrumpfs)

Beispiel:

```
1 for j in "Hallo":
2     print(j* 2)
```

```
HH
aa
ll
ll
oo
```

```
1 for ingredient in ("sugar", "water", "oil"):
2     if ingredient == "sugar":
3         print("sweet!")
```

```
sweet!
```

Ablauf der Schleife beeinflussen

- **break** - Schleifenrumpf wird umgehend beendet (eventuell vorzeitig)
- **continue** - die aktuelle Schleifeniteration wird vorzeitig beendet, es wird in den Schleifenkopf gesprungen und die Schleifenvariable wird auf den nächsten Wert gesetzt.
- **else-Zweig** - wird nach Beendigung der Schleife ausgeführt, genau dann wenn die Schleife nicht mit **break** verlassen wurde.

Beispiel

```
1 noten_und_anzahl = [("sehr gut", 1), ("gut", 3), ("befriedigend",0), ("ausreichend", 2)]
2
3 for na in noten_und_anzahl:
4     note , anzahl = na
5     if anzahl == 0:
6         continue
7     if note == "h":
8         print("Tolle Leistung")
9         break
10 else:
11     print("Durchschnitt war in Ordnung")
12
```

Durchschnitt war in Ordnung

Alternativ kann man das auch so aufschreiben:

```
1 noten_und_anzahl = [("sehr gut", 1), ("gut", 3), ("befriedigend",0), ("ausreichend", 2)]
2
3 for note, anzahl in noten_und_anzahl:
4     if anzahl == 0:
5         continue
6     if note == "gut":
7         print("Tolle Leistung")
8         break
9 else:
10     print("Durchschnitt war in Ordnung")
11
```

Tolle Leistung

Nützliche Funktionen

- range
- zip
- reversed

12.2 range

- erzeugt eine Folge von Indices für die Schleifendurchläufe, genauer einen Iterator.
- range(stop) - ergibt 0, 1, ..., stop -1
- range(start, stop) - ergibt start, start + 1, ... , stop -1
- range(start, stop, step) - ergibt start, start +step, start+ 2* step, ... , stop -1

Beispiele:

```
1 range(5)
```

range(0, 5)

```
1 print(range(2, 30, 5)) #keine Liste, print gibt keine Ausgabe
```

range(2, 30, 5)

```
1 list(range(0,9))
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8]
```

```
1 list (range(2, 30, 5))
```

```
[2, 7, 12, 17, 22, 27]
```

```
1 for i in range(3,6):  
2     print (i, "**3 = ", i ** 3)
```

```
3 **3 = 27  
4 **3 = 64  
5 **3 = 125
```

12.3 zip

- Funktion
- nimmt eine oder mehrere Sequenzen und liefert eine "Liste" von Tupeln mit korrespondierenden Elementen
- erzeugt eigentlich keine Liste, sondern einen Iterator

Beispiel

```
1 fleisch = ["Rind", "Huhn", "Hund"]  
2 beilage = ["Kartoffeln", "Pommes", "Spätzle"]  
3  
4 print(list(zip(fleisch, beilage)))
```

```
[('Rind', 'Kartoffeln'), ('Huhn', 'Pommes'), ('Hund', 'Spätzle')]
```

- mit zip können mehrere Sequenzen parallel durchlaufen werden
- bei unterschiedlicher Länge ist das Ergebnis so lang, wie die kürzere Sequenz.

Beispiel

```
1 for xyz in zip("rind", "huhn", range(5,10)):  
2     x, y, z = xyz  
3     print(x, y, z)
```

```
r h 5  
i u 6  
n h 7  
d n 8
```

12.4 reversed

- Funktion
- Durchlauf einer Sequenz in umgekehrter Richtung

Beispiel

```
1 for x in reversed("hallo"):
2     print(x)
```

```
o
l
l
a
h
```

12.5 Beispiel: Implementierung einer Fakultätsfunktion

Fakultät ist wie folgt definiert:

$$0! = 1$$

$$(n + 1)! = (n + 1) \cdot n!$$

Aufgabe: Entwickle eine Funktion `fact`, welche die Fakultät einer ganzen Zahl berechnet.

1. Bezeichner und Datentypen

- Die Eingabe ist `n: int` (mit $n \geq 0$)
- Die Ausgabe ist `int`

2. Funktionsgerüst python

```
# fill in         return
```

```
def fact(n: int)-> int ##assume n>1
```

3. Beispiele

```
assert fact(0) == 1
assert fact(1) == 1
assert fact(3) == 6
```

4. Ergebnis

```
1 def fact(  
2     n: int  
3     ) -> int:  
4     result = 1  
5     for i in range (1, n+1):  
6         result= result * i  
7     return result
```

```
1 print(fact(10))
```

3628800